

Advanced Natural Language Processing Distillation Compression Pruning

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes and Dr. Aki Harma – Course year 2026-2027



Maastricht University

Cost in Energy for basic pre-training:

- GPT-2: 100-200k
- GPT 3: estimated 1-10 mio
- GPT 4: estimated 10-100 mio

Estimation for a custom GPT-4 model is now 2-3 million.



Menu

Information

The Custom Models program gives selected organizations an opportunity to work with a dedicated group of OpenAI researchers to train custom GPT-4 models to their specific domain. This includes modifying every step of the model training process, from doing additional domain specific pre-training, to running a custom RL post-training process tailored for the specific domain. Organizations will have exclusive access to their custom models. This program is particularly applicable to domains with extremely large proprietary datasets—billions of tokens at minimum.

Inquire

☐ I acknowledge that it may take several months to train custom models, and that pricing starts at \$2–3 million

Submit

Computational Complexity of Transformers (and all other models in the Transformer Zoo)

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

n = sequence length (40-70),
 d = depth (number layers),
 k = kernel size of the convolution,
 d = length word embedding (128-512)
 r = attention window size

Model	Hardware	Power (W)	Hours	kWh-PUE	CO ₂ e	Cloud compute cost
Transformer _{base}	P100x8	1415.78	12	27	26	\$41–\$140
Transformer _{big}	P100x8	1515.43	84	201	192	\$289–\$981
ELMo	P100x3	517.66	336	275	262	\$433–\$1472
BERT _{base}	V100x64	12,041.51	79	1507	1438	\$3751–\$12,571
BERT _{base}	TPUv2x16	—	96	—	—	\$2074–\$6912
NAS	P100x8	1515.43	274,120	656,347	626,155	\$942,973–\$3,201,722
NAS	TPUv2x1	—	32,623	—	—	\$44,055–\$146,848
GPT-2	TPUv3x32	—	168	—	—	\$12,902–\$43,008

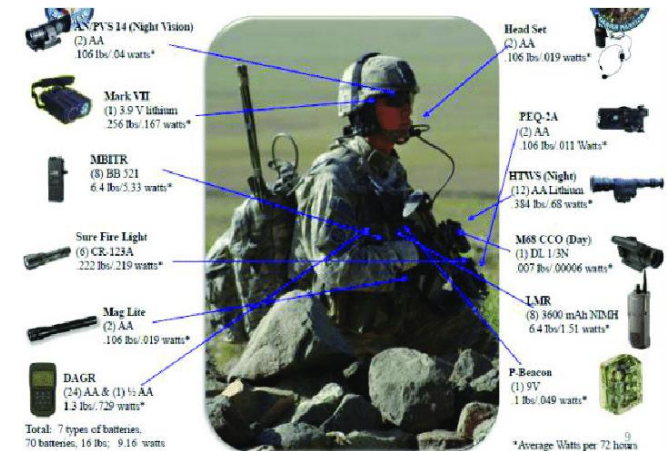
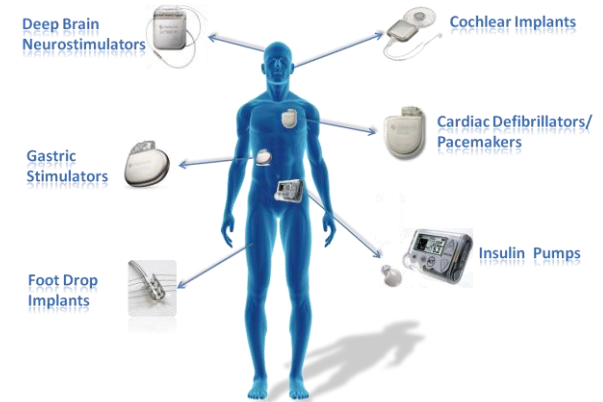
Table 3: Estimated cost of training a model in terms of CO₂ emissions (lbs) and cloud compute cost (USD).⁷ Power and carbon footprint are omitted for TPUs due to lack of public information on power draw for this hardware.

Consider more efficient AI: when battery life matters ...

- Implanted Medical Devices
- Mobile devices
- Smart Watch
- Military devices
- ...



WIRELESS IMPLANTABLE MEDICAL DEVICES



DistilBERT

- Operating these large NLP Deep Learning models in on-the-edge and/or under constrained computational training or inference budgets remain challenging.
- DistilBERT is smaller general-purpose language representation model.
- DistilBERT can then be fine-tuned with good performances on a wide range of tasks like its larger counterparts.
- It is possible to reduce the size of a BERT model by 40%, while retaining 97% of its language understanding capabilities and being 60% faster.

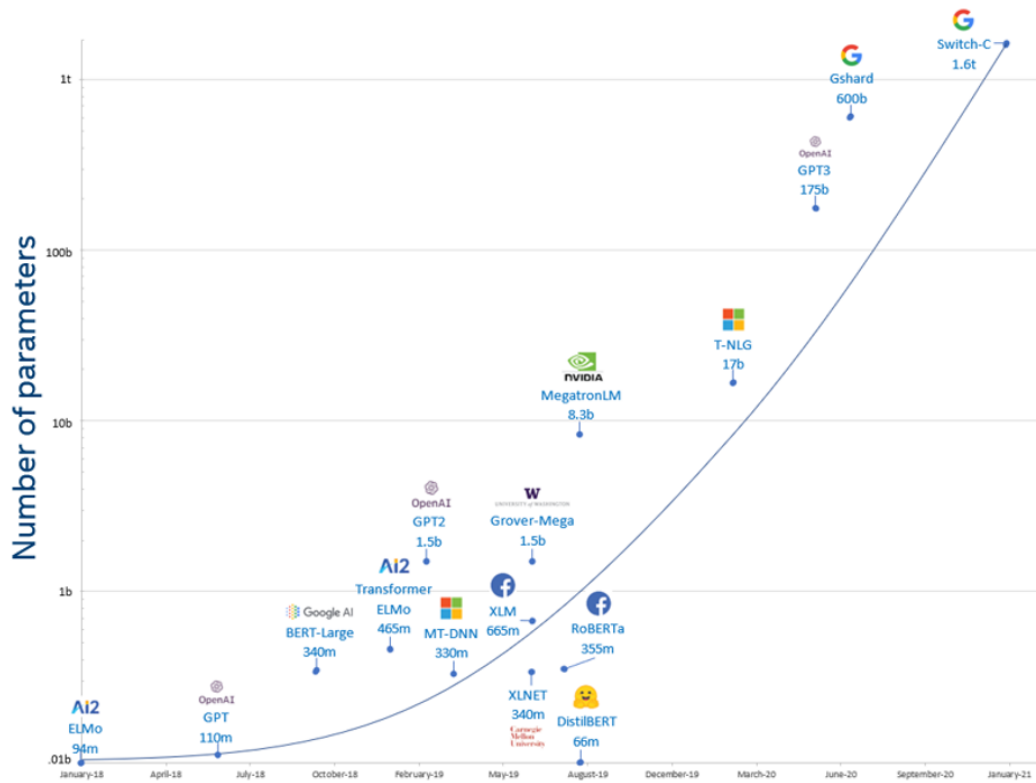


Figure 1: Exponential growth of number of parameters in DL models

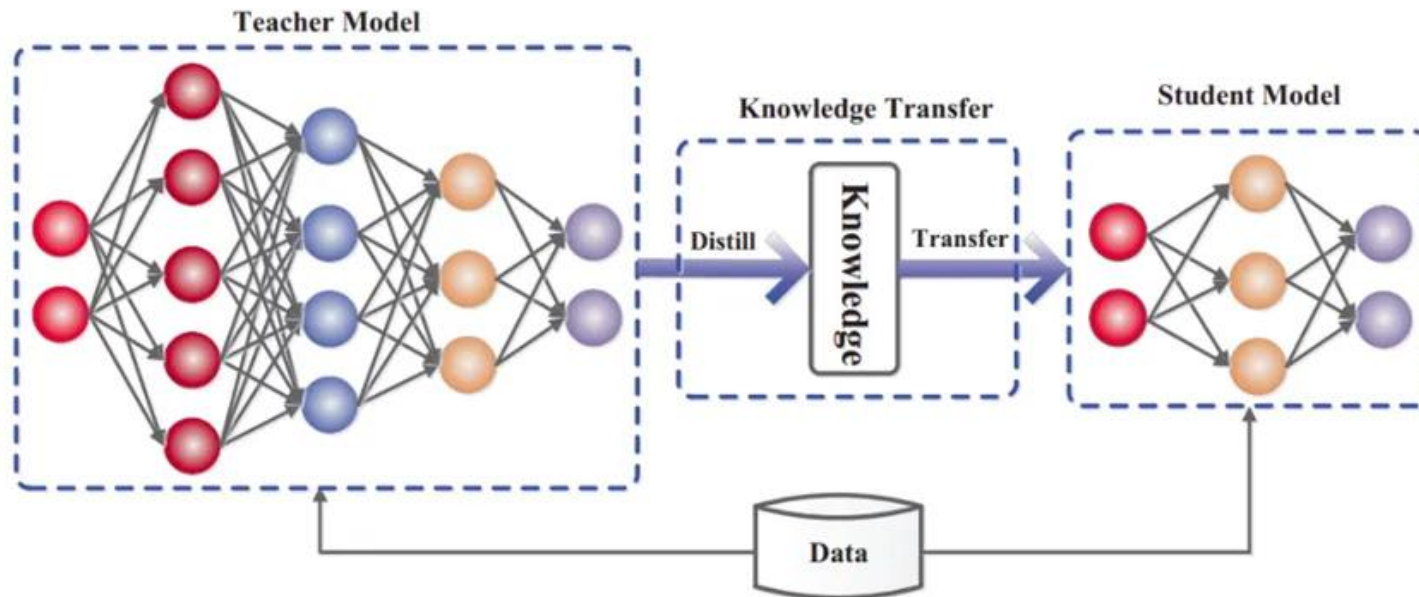
Table 1: **DistilBERT retains 97% of BERT performance.** Comparison on the dev sets of the GLUE benchmark. ELMo results as reported by the authors. BERT and DistilBERT results are the medians of 5 runs with different seeds.

Model	Score	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
ELMo	68.7	44.1	68.6	76.6	71.1	86.2	53.4	91.5	70.4	56.3
BERT-base	79.5	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	77.0	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

Sanh, Victor & Debut, Lysandre & Chaumond, Julien & Wolf, Thomas. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.

Knowledge Distillation in DistillBERT

Knowledge distillation refers to the process of compressing and transferring knowledge from a computationally expensive large model(teacher model) to a smaller model(student model) by maintaining validity.



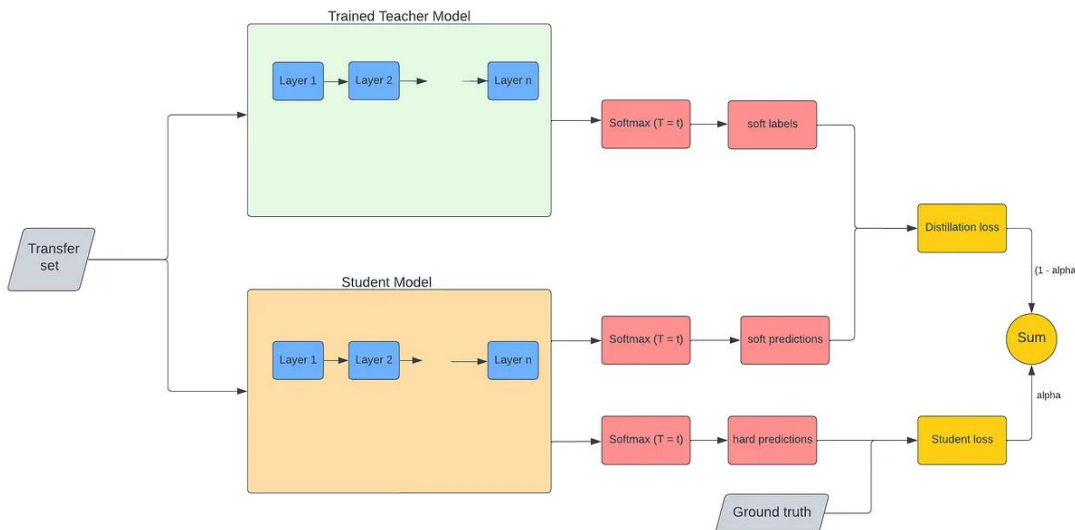
(Teacher-Student model for Knowledge Distillation)

Knowledge Distillation Training

For distillation we use another set of training data (which is not in training data used for training teacher model) called transfer set. Actually, here the forward pass is done through pretrained teacher model and student model and the loss is computed as below.

$$\text{total loss} = (\alpha * \text{Student model loss}) + ((1 - \alpha) * \text{distillation loss})$$

Here alpha is the factor which weights student loss and distillation loss.



For calculating distillation loss we use KL-divergence loss. For that we take the predictions of teacher model with $t > 1$ and predictions of student model with $t > 1$. The output of these SoftMax is the soft labels.

$$D_{KL}(p(x) || q(x)) = \sum_{x \in X} p(x) \ln \frac{p(x)}{q(x)}$$

Other approaches to make LLM more efficient

- TinyBERT: transfer the (plenty) of knowledge encoded in large-BERT to a small student tinyBERT.
- DistilBERT: DistilBERT is a small, fast, cheap and light Transformer model trained by distilling BERT base. It has 40% less parameters than *bert-base-uncased*, runs 60% faster while preserving over 95% of BERT's performances as measured on the GLUE language understanding benchmark
- Quantization: e.g.: dynamic quantization converts a floating-point model to a quantized model with static int8 or float16 data types for the weights and dynamic quantization for the activations. No need for 64 bit floats in your weights, 8 or 16 bit will do. Leads easily to 25-250% speed up (depends on hardware)
- Pruning: An alternative to quantization is pruning. Pruning introduces zeros (aka sparsity) in the weight matrices, leading to more efficient matrix calculations in terms of both memory and computation. One can either prune “weights” or entire “neurons”.
- ... this is an on-going field of research ...

NEWS | 05 October 2022

DeepMind AI invents faster algorithms to solve tough maths puzzles

Machine-learning technique improves computing efficiency and could have far-reaching applications.

KV caching , Flash-attention, etc



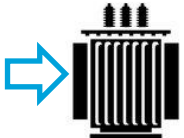
Maastricht University

Autoregressive generation

$$y = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

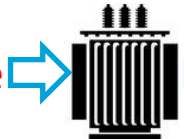
$$Q = W_Q X, K = W_K X, \text{ and } V = W_V X$$

$$X_k = [\bar{x}_0, \bar{x}_1, \dots, \bar{x}_k, \bar{0}, \dots]$$

The river in Maastricht is called  Je

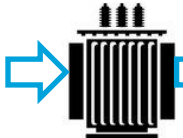
update X

$$X_{k+1} = [\bar{x}_0, \bar{x}_1, \dots, \bar{x}_k, \bar{x}_{k+1}, \bar{0}, \dots]$$

The river in Maastricht is called Je  ke

update X

$$X_{k+2} = [\bar{x}_0, \bar{x}_1, \dots, \bar{x}_k, \bar{x}_{k+1}, \bar{x}_{k+2}, \bar{0}, \dots]$$

The river in Maastricht is called Jeke  r.

update X

$$X_{k+3} = [\bar{x}_0, \bar{x}_1, \dots, \bar{x}_k, \bar{x}_{k+1}, \bar{x}_{k+2}, \bar{x}_{k+3}, \bar{0}, \dots]$$

Insight: in every step, we just update one token vector in X but then compute all (huge) Matrix operations for the next token. Can't we optimize this?

KV Cache

$$y = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

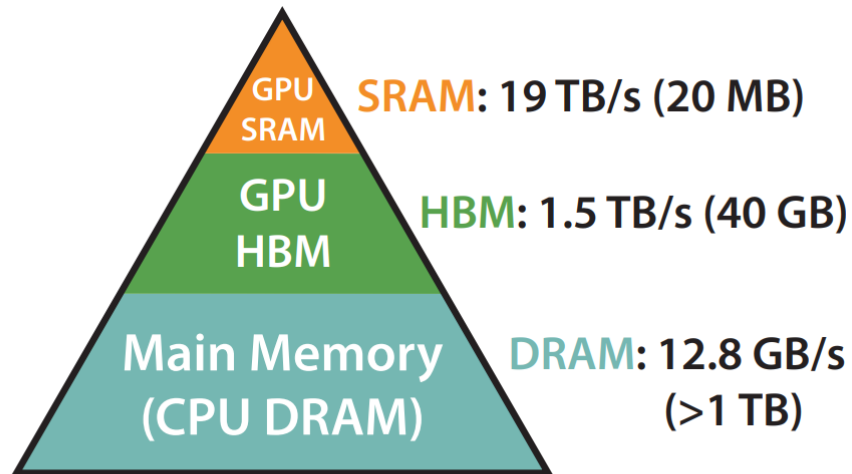
$$Q = W_Q X, K = W_K X, \text{ and } V = W_V X$$

- It is enough to cache (and accumulate) K and V matrices – much less computation!
- The cache still takes a lot of memory (inside GPU)
- KV Cache compression methods
 - Quantization
 - Eviction of low-impact values
 - Predictive modeling
 - Cache merging

Flash Attention – I/O-efficient inference

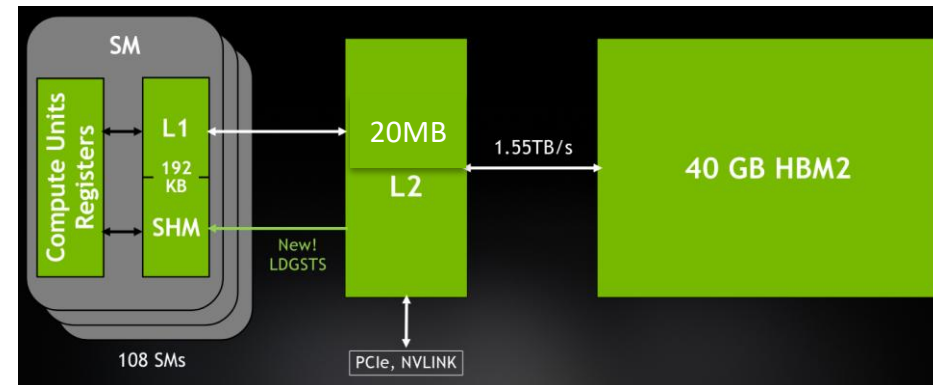
- The speed of the memory access has a large impact on the speed of self-attention computation
- GPU SRAM access is fast but limited size
- HBM (High Bandwidth Memory) is big but slow

FlashAttention – addressing an architecture issue



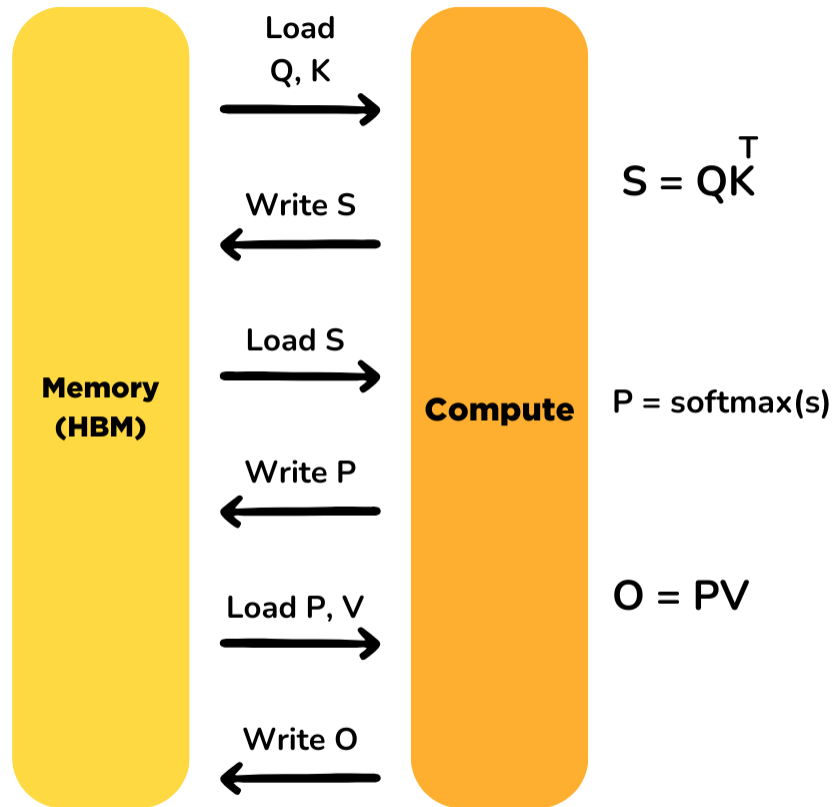
Memory Hierarchy with
Bandwidth & Memory Size

NVIDIA A100 40G GPU

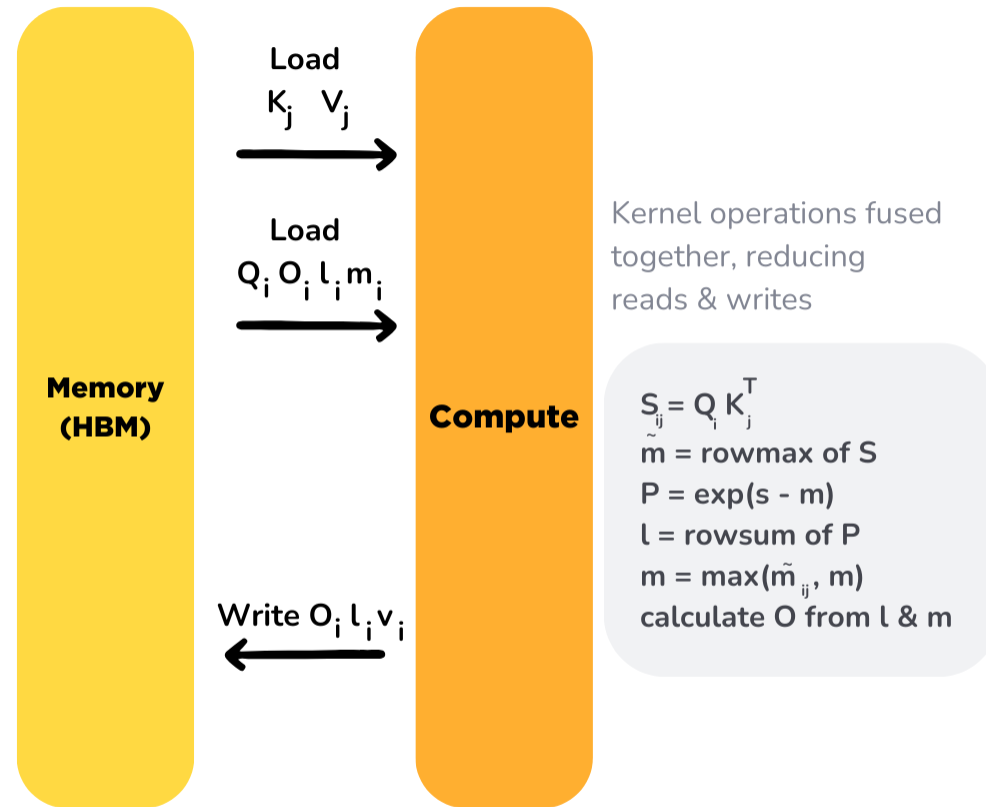


[\[2205.14135\] FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#)

Standard Attention Implementation



Flash Attention



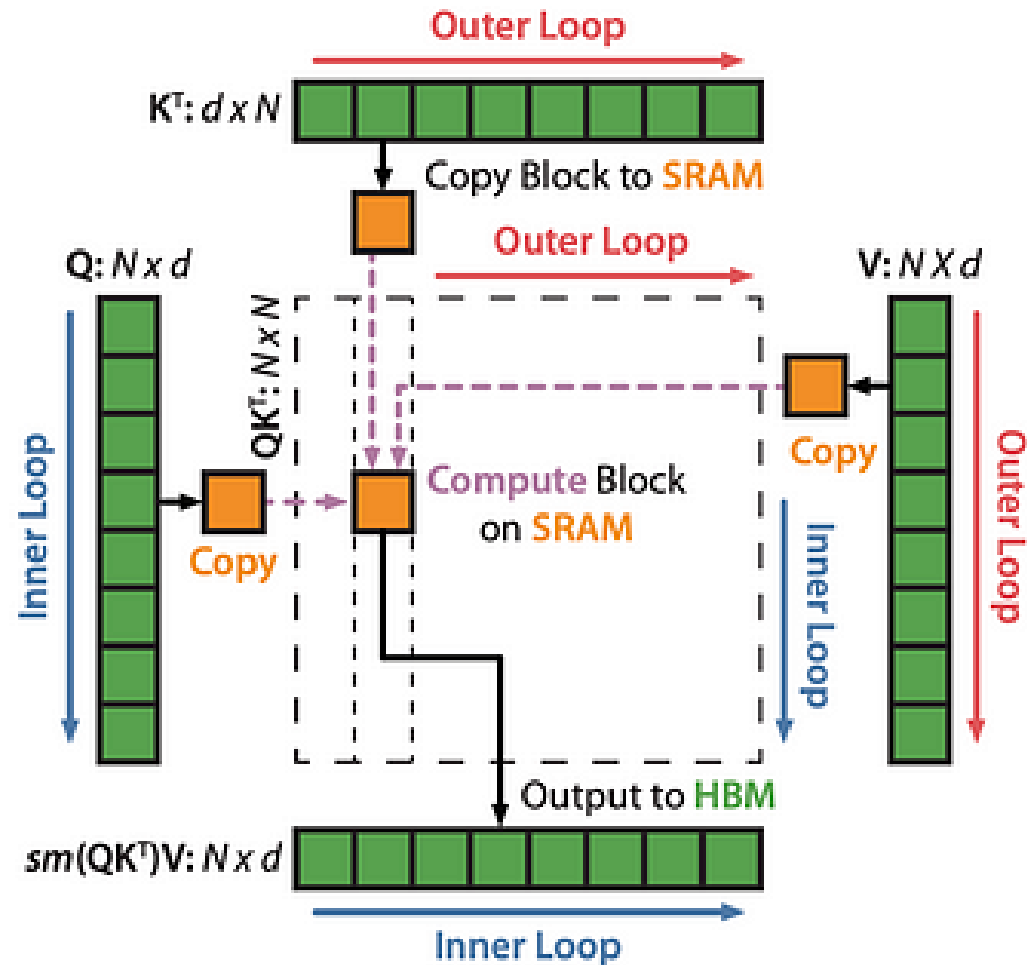
Initialize O , l and m matrices with zeroes. m and l are used to calculate cumulative softmax. Divide Q , K , V into blocks (due to SRAM's memory limits) and iterate over them, for i is row & j is column.

Tiling the matrices

$$y = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = W_Q X, K = W_K X, \\ \text{and } V = W_V X$$

Reorganize and slice

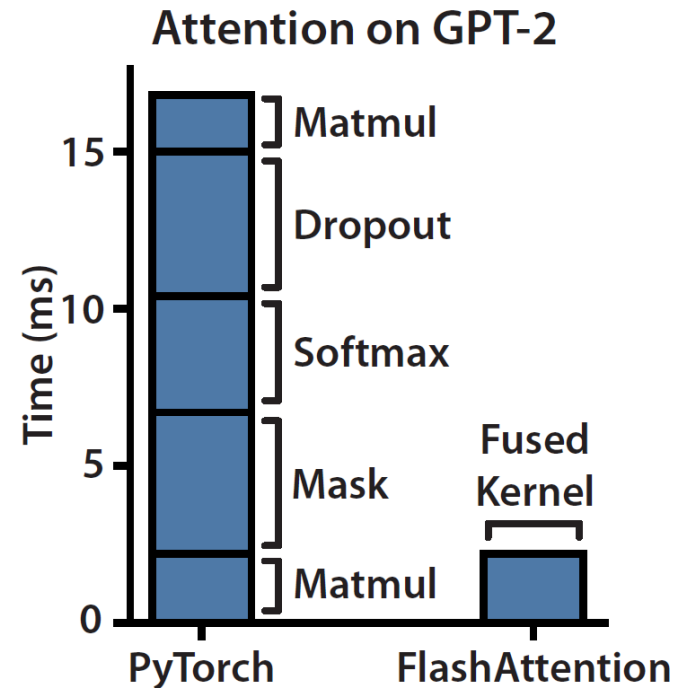


Tiling matrix operations and recomputation

$$y = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = W_Q X, K = W_K X, \text{ and } V = W_V X$$

Further improvements in
FlashAttention-2,
FlashAttention-3



Data: training of large models



Common Crawl
maintains a **free, open repository** of web crawl
data that can be used by
anyone.

Common Crawl is a 501(c)(3) non-profit founded in 2007.

We make wholesale extraction, transformation and analysis of
open web data accessible to researchers.

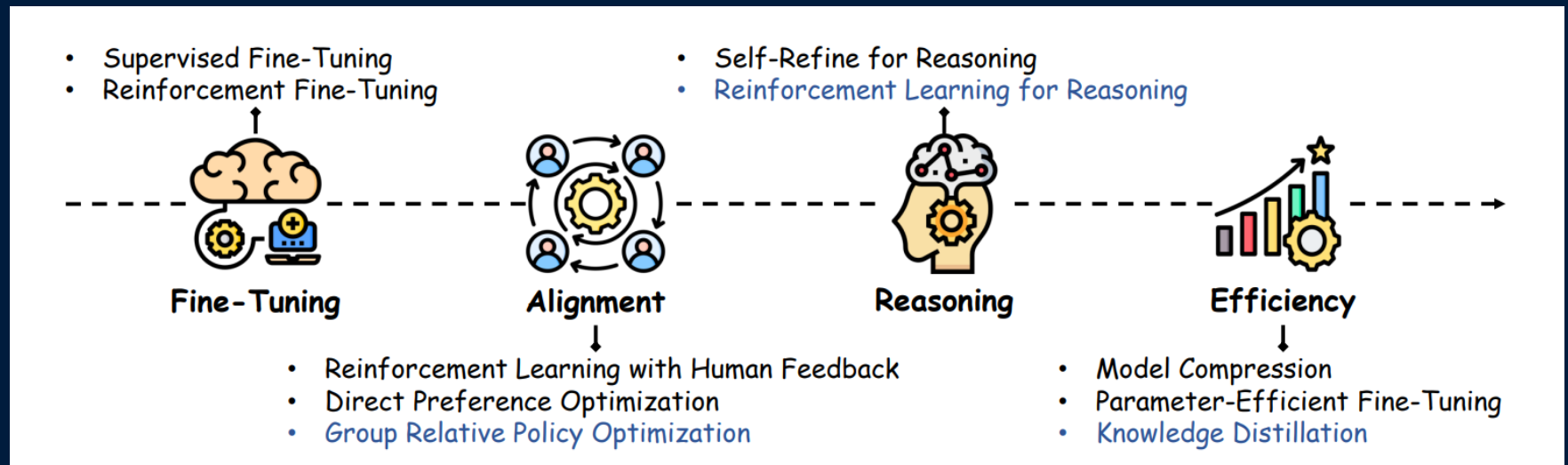
[Overview](#)

Over **250 billion** pages spanning 17 years.

Free and open corpus since 2007.

Cited in over **10,000** research papers.

Post-training



LLMs

- Pretrained one-size-fits-all model
- A large variety of derived models with different Post-trainings

<https://arxiv.org/pdf/2503.06072>

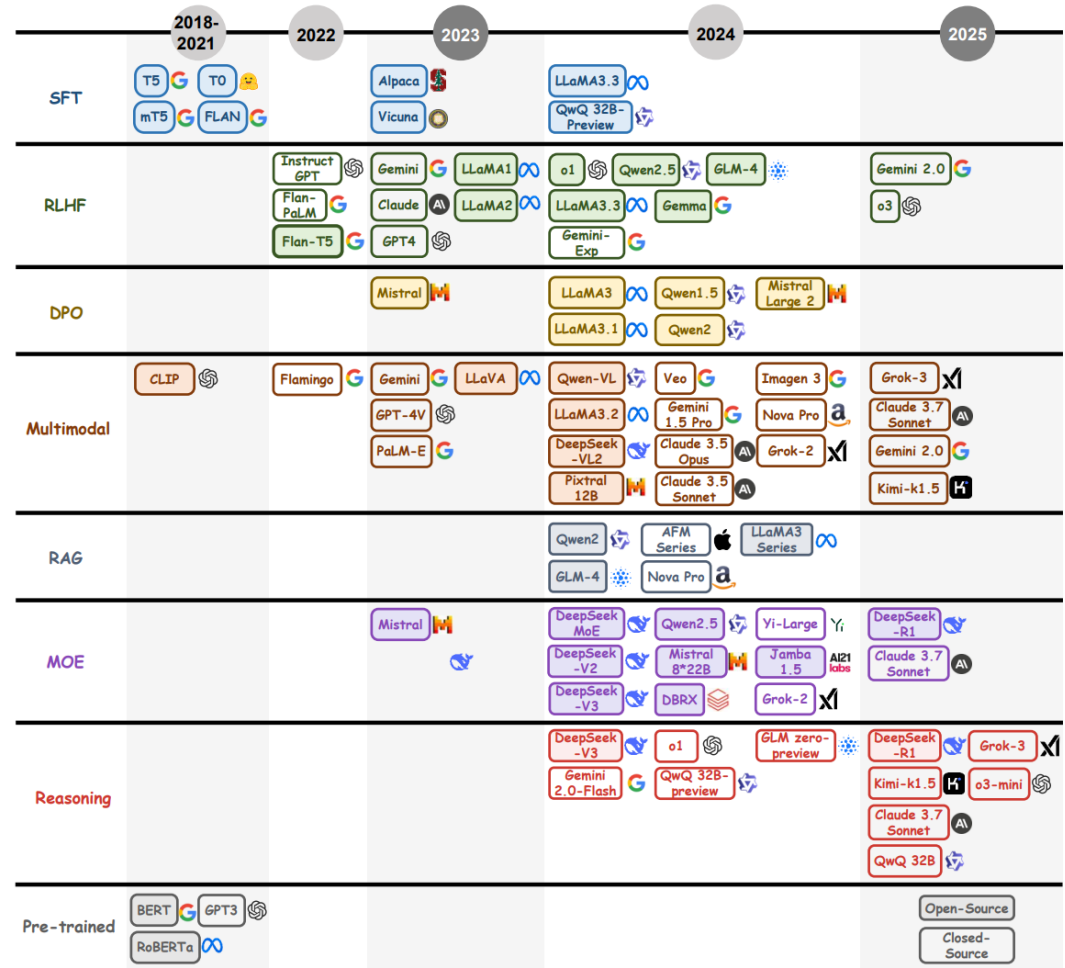
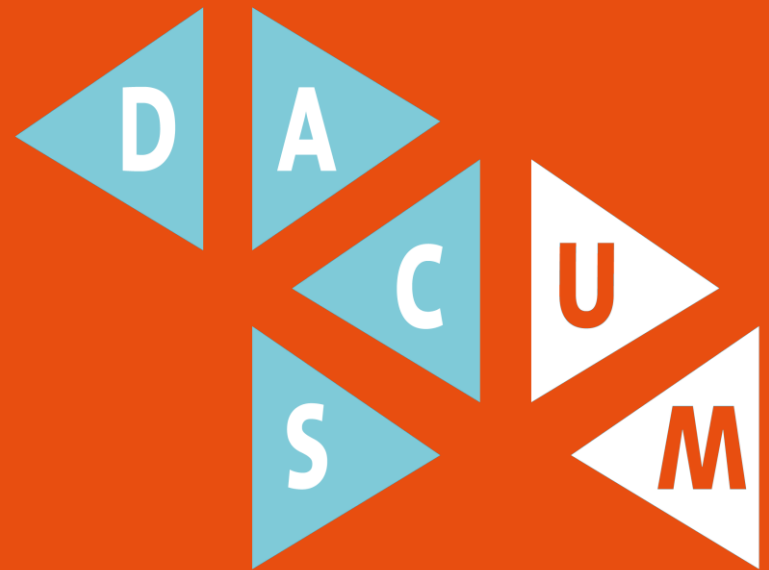


Figure 3: Timeline of post-training technique development for Large Language Models (2018–2025), delineating key milestones in their historical progression.

Efficiency: optimization of LLM inference



Improving efficiency

- Algorithmic optimizations (KV Cache, etc.)
- Memory access optimization (Flash Attention)
- Quantization
- Low-Rank Adaptation (LoRA)
- Distilling into smaller models
- Better training and data
- New architectures

Quantization

- 32-bit Floating Point arithmetic
- If we can represent all numbers by 8 bits
 - Memory size is $\frac{1}{4}$ of the original
 - Data transfer is 4-times faster
 - Computation can be much faster (especially integer arithmetic)
- Challenges
 - Quantization errors
 - Over/underflow in operations

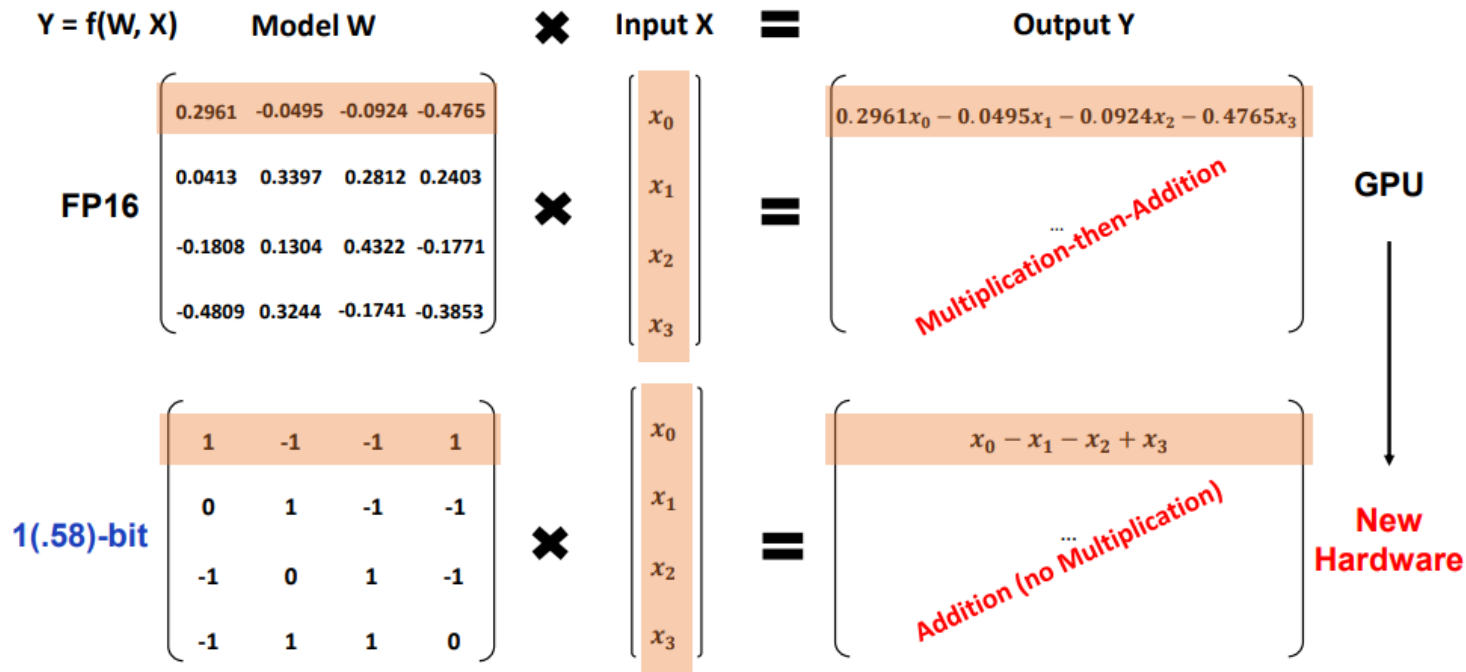
$$Q = W_Q X, K = W_K X, \text{ and } V = W_V X$$

$$y = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Post-Training Quantization (PTQ)

Quantization-Aware Training (QAT)

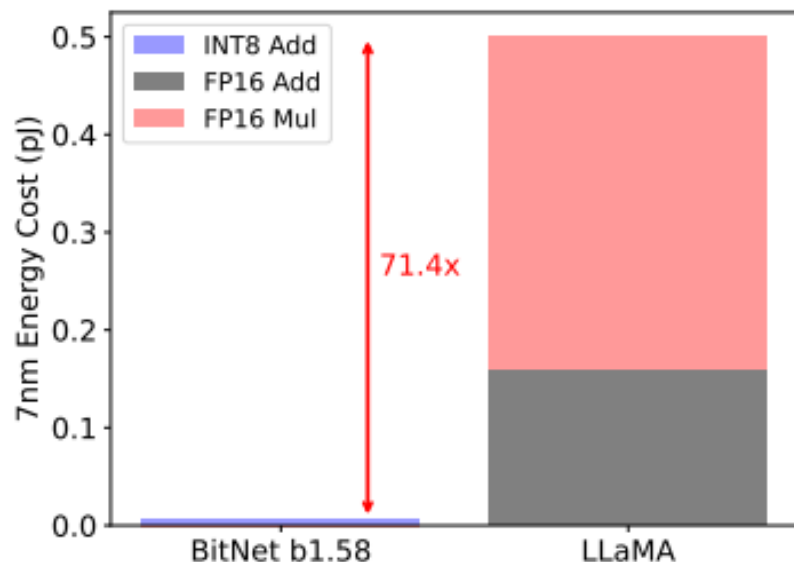
QAT Example: BitNet b1.58, i.e., coefficients in $\{-1, 0, 1\}$



Same performance as the base LLaMA model, much lower requirements (runs in any CPU)

<https://arxiv.org/abs/2402.17764>

QAT Example: BitNet b1.58, i.e., coefficients in $\{-1, 0, 1\}$



Similar performance as the base model, but a much lower energy consumption: 1/71.4!!

Models	Tokens	Winogrande	PIQA	SciQ	LAMBADA	ARC-easy	Avg.
StableLM-3B	2T	64.56	76.93	90.75	66.09	67.78	73.22
BitNet b1.58 3B	2T	66.37	78.40	91.20	67.63	68.12	74.34

Table 4: Comparison of BitNet b1.58 with StableLM-3B with 2T tokens.

<https://arxiv.org/abs/2402.17764>

Low-Rank Adaptation (LoRA)

$Q = W_Q X$, where W_Q , a $M \times N$, matrix can be approximated by

$W_Q \approx A_q B_q^T$, where

A_q is $M \times d$ matrix and B_q is a $N \times d$ matrix



If $d \ll M, N$, the number of coefficients/computations
 $(M + N)d \ll MN$

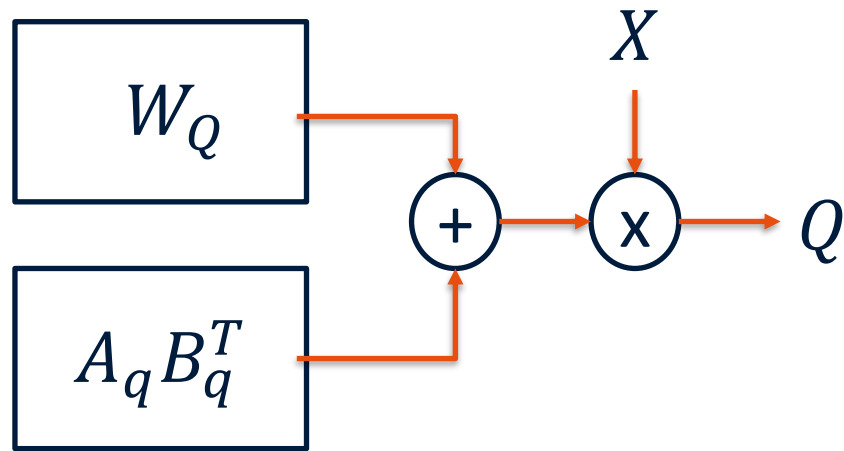
GPT-3: $M = 12244, N = 128$. If $d = 4$,
 $MN = 1567232$ but $(M + N)d = 49488$ (3% parameters !)

Low-Rank Adaptation (LoRA)

We cannot just replace all W_Q and W_V by some random matrices and start training the model with some other data! This also is an adapter (see above)

$$y = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = W_Q X, K = W_K X, \\ \text{and } V = W_V X$$



Start of fine-tuning: A initialized as a random matrix, B initialized by zeros, all original weights W_Q frozen

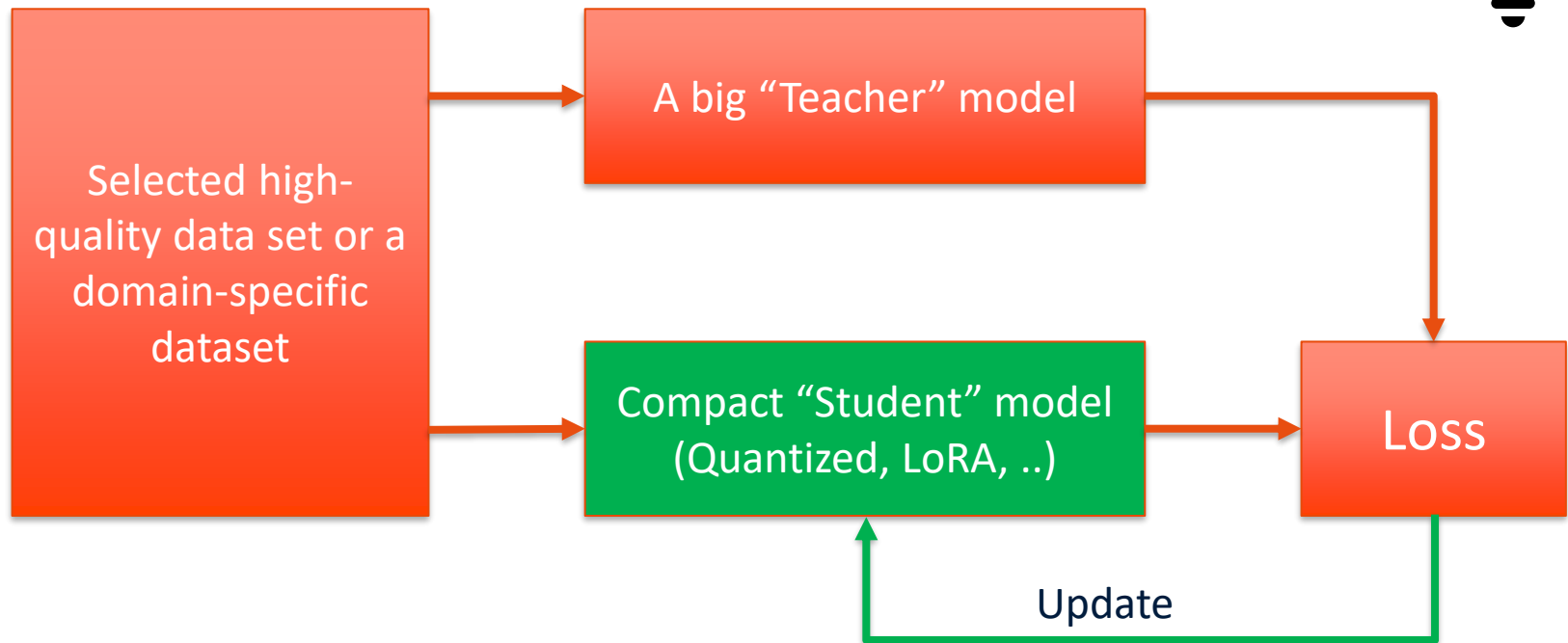
Low-Rank Adaptation (LoRA)

- After the fine tuning, we may update the attention weights $W_Q = W_Q + A_d B_d^T$ and use the new model
- Usually applied to W_Q and W_V matrices at all layers
- The rank d is typically 4-8

Quantized Low-Rank Adaptation (QLoRA)

- The coefficients in A and B are also quantized (faster training)

Distillation, specialization, PTQ



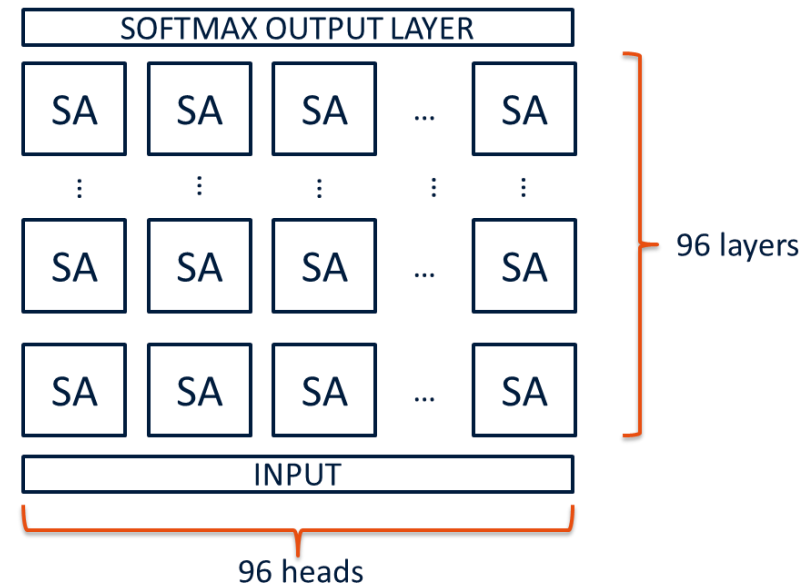
Quadratic scaling of transformers

$$X = (x_0, x_1, \dots, x_{N-1}), V = W_V X$$

Maximum sequence length N

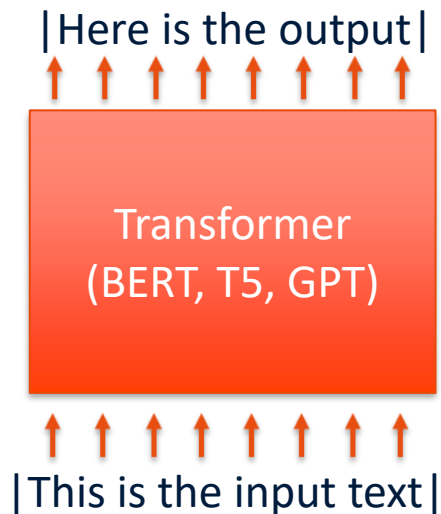
$$y = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$N \times N$ matrix

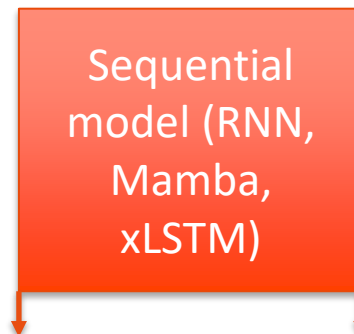


Coefficients $\sim N^2$ in every head and layer for any input text, like, "Hello!?"

Block-based and sequential architectures



| Fixed model size |

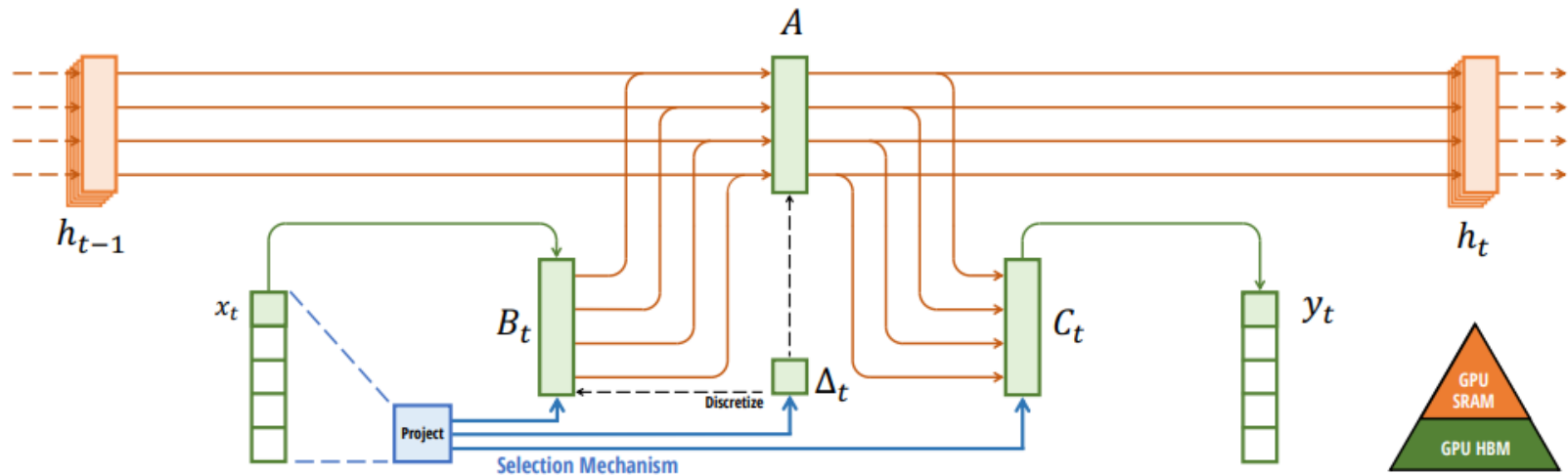


The input can be of any length, the model scans over...

Alternative architectures

- Decouple the model size and sequence

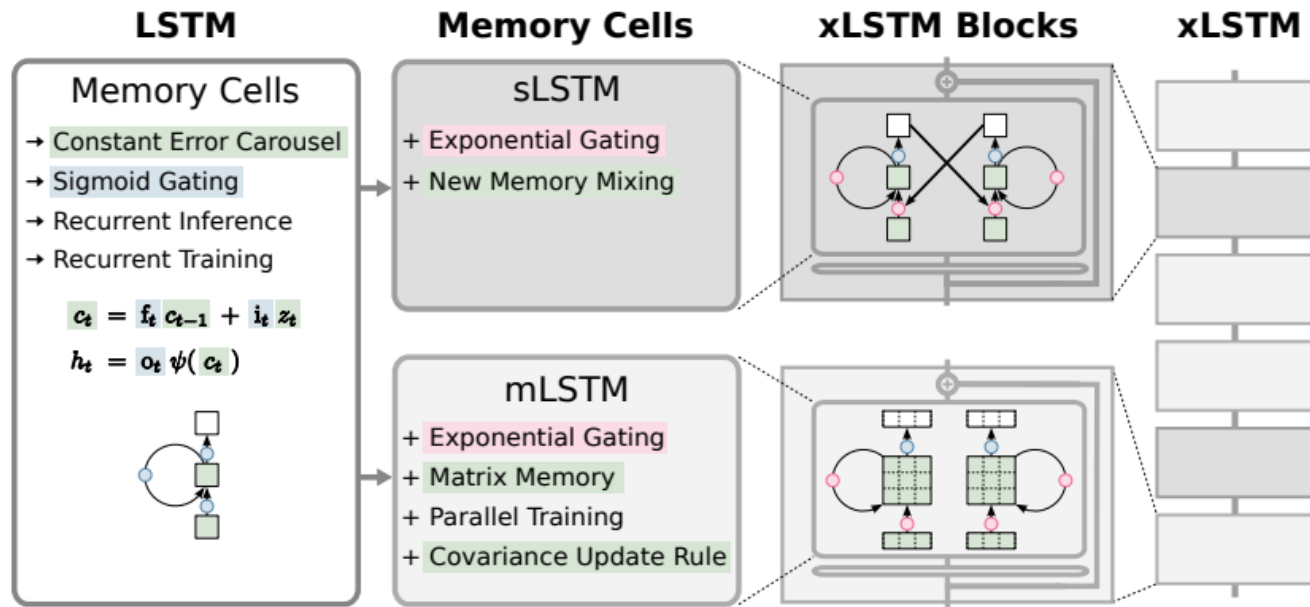
Selective State Space Model
with Hardware-aware State Expansion



[2312.00752] Mamba: Linear-Time Sequence Modeling with Selective State Spaces

Alternative architectures

- xLSTM architecture (evolved from RNNs)



[\[2405.04517\] xLSTM: Extended Long Short-Term Memory](#)

Adversarial Defense & Distillation/Pruning

Utilize adversarial examples to enhance the model's robustness by:

- Adversarial Training: use various forms of augmentation, add hand-crafted negations, synonyms, train with noisy data, utilize dropout to reduce overfitting, etc.
- Knowledge distillation: compress the model and remove non-used connections that can be abused. Aka feature squeezing. We will discuss that in the next section.
- Leverage an ensemble of different machine learning models.



Questions?



Maastricht University

Exercise 1: Knowledge Distillation Loss and Soft Labels (DistilBERT)

Knowledge distillation transfers representations from a large teacher model to a compact student network by training the student on both hard ground-truth labels and smoothed "soft labels" generated by the teacher.

- In the lecture, we specify the total loss function for training DistilBERT:

$$\text{total loss} = \alpha \cdot \mathcal{L}_{\text{student}} + (1 - \alpha) \cdot \mathcal{L}_{\text{distillation}}$$

Identify what each loss term represents, and explain why Kullback-Leibler (KL) divergence is chosen for $\mathcal{L}_{\text{distillation}}$.

- The soft predictions and soft labels are generated using softmax with temperature $T > 1$:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Why does scaling logits by temperature $T > 1$ expose rich "dark knowledge" (e.g., secondary class probabilities) that standard one-hot hard labels completely erase?

- According to Sanh et al. (2019), what percentage of parameters and latency is reduced in DistilBERT compared to Bert-base-uncased, and how much of BERT's original GLUE performance is retained?



Exercise 2: Parameter-Efficient Fine-Tuning via LoRA and Quantization

Low-Rank Adaptation (LoRA) and quantization techniques allow adaptation and compression of multi-billion parameter models under tight hardware constraints.

- In LoRA (slides 23–24), an original frozen projection matrix $W_Q \in \mathbb{R}^{M \times N}$ is updated using low-rank decomposition matrices A_q and B_q :
$$W_Q \leftarrow W_Q + A_q B_q^T$$
 - If $M = 4096$, $N = 4096$, and rank $r = 8$, calculate the number of trainable parameters in $A_q B_q^T$ versus W_Q . What is the parameter reduction ratio?
 - How are A_q and B_q initialized at the beginning of training to ensure the model's pre-trained behavior is initially unaltered?
- Contrast **Post-Training Quantization (PTQ)** with **Quantization-Aware Training (QAT)**. Why does reducing weights from 32-bit/16-bit floating point to 8-bit integers (or 1.58-bit ternary values in BitNet b1.58) lead to significant energy savings and hardware speedups?
- Explain the difference between weight pruning and neuron/head pruning. How does introducing structured sparsity assist in edge deployment for resource-constrained environments (e.g., wearables or medical implants)?

Exercise 3: Hardware-Aware Attention Optimization (KV Caching & FlashAttention)

During autoregressive decoding, token-by-token generation faces major memory bandwidth and compute bottlenecks.

- In the lecture, we demonstrate that during autoregressive generation of token x_{k+1} , previously generated tokens $x_0, \dots, x_k$ remain fixed.
 - Explain what **KV Caching** stores across decoding steps.
 - Why does caching Key and Value projections eliminate redundant matrix multiplications, and what memory challenge does it introduce inside GPU memory as context length grows?
- Using the GPU memory hierarchy shown on slides 12–14 (SRAM vs. HBM):
 - Contrast the memory size and bandwidth of on-chip SRAM versus High Bandwidth Memory (HBM) on an NVIDIA A100.
 - In standard attention, why do intermediate reads and writes of the $N \times N$ attention matrix ($S = QK^T$ and $P = \text{softmax}(S)$) to HBM create an I/O bottleneck?
- How does **FlashAttention** overcome this memory traffic bottleneck using kernel fusion and matrix tiling? Why does computing exact softmax over tiled blocks entirely within fast SRAM without materializing the full attention matrix in HBM achieve such drastic wall-clock speedups?

Discussion



On Distillation and the Origin of Dark Knowledge: Compression or Mimicry?

In knowledge distillation, a compact student model (like DistilBERT) shrinks parameter count by 40% and speeds up inference by 60% while retaining over 95% of the teacher's benchmark performance by training on soft probability labels scaled by temperature $T > 1$.

These soft targets reveal 'dark knowledge'—the subtle, non-zero secondary probabilities across the entire vocabulary that one-hot ground-truth labels completely erase.

If a student with nearly half the layers and parameters can match the teacher's decisions almost perfectly, does that prove the original teacher model was massively overparameterized and inefficient?

Or did the student merely memorize the surface distribution of the teacher's outputs without internalizing the deep, generalizable abstractions that required hundreds of millions of parameters to learn in the first place?

If knowledge can be distilled with negligible performance loss, why do we need to train massive foundation models from scratch at all?

On FlashAttention and the Hardware Illusion: Algorithm vs. Silicon

For years, the quadratic $O(N^2)$ complexity of self-attention was treated as a purely mathematical problem in algorithm design, prompting researchers to build complex sparse or low-rank approximations that compromised model quality.

Yet FlashAttention achieved dramatic wall-clock speedups without altering the mathematical formula of attention at all—it simply rearranged memory traffic between high-speed on-chip SRAM (20 MB at 19 TB/s) and slow High Bandwidth Memory (HBM, 40 GB at 1.5 TB/s) using matrix tiling and kernel fusion.

If the biggest computational bottleneck in modern NLP was not the number of arithmetic operations, but the physical time spent moving tensors back and forth across GPU silicon buses, is theoretical algorithmic complexity ($O(N^2)$) an accurate measure of computational efficiency in deep learning?

How much of our NLP progress is driven by linguistic and mathematical innovation, and how much is merely low-level hardware co-design and memory cache management?



On the Return of the Sequence: Can Attention Ever Scale to Infinity?

The lecture illustrates that Transformers require computing an $N \times N$ attention matrix across 96 layers and 96 heads for every input, meaning context expansion inevitably hits a quadratic wall in memory and energy.

In response, cutting-edge architectures like Mamba (Selective State Space Models) and xLSTM (Extended LSTM) are reviving recurrent, sequential processing—decoupling sequence length from model memory to achieve true linear-time $O(N)$ scaling.

In earlier lectures (History of Deep Learning), we concluded that recurrent models were structurally obsolete because they suffered from information bottlenecks and could not be parallelized on GPUs.

If modern state-space and recurrent models can now be trained in parallel while matching Transformer performance at a fraction of the compute, was the historical shift to pure attention just a temporary detour created by the limitations of early 2010s hardware?

Will the future of NLP belong to quadratic attention or linear state spaces?